

Query Optimization

Query:
SELECT...
FROM...
WHERE...



Query
Optimizer



Query Plan

Database Schema,
Distribution Statistics

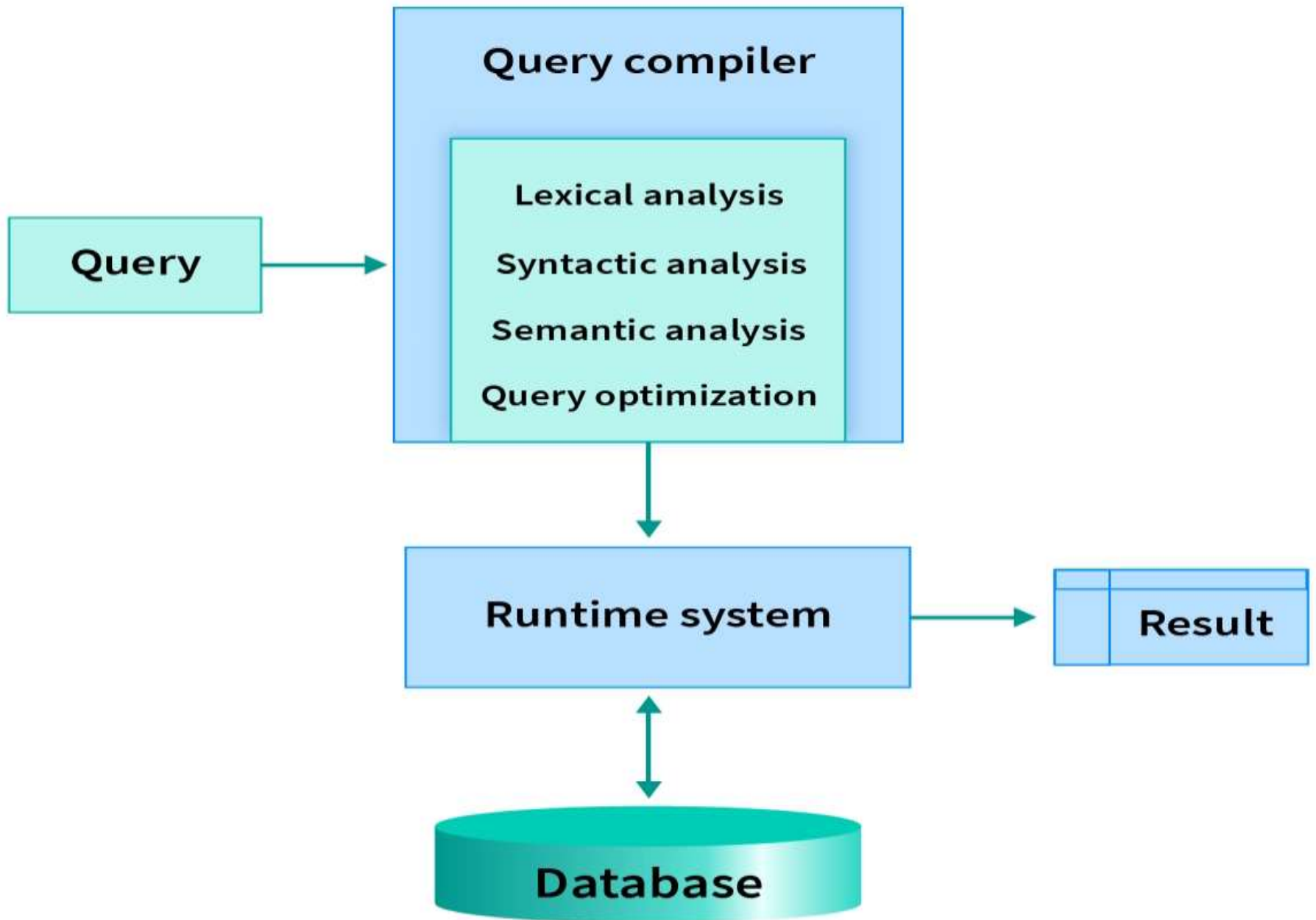
Query Processing

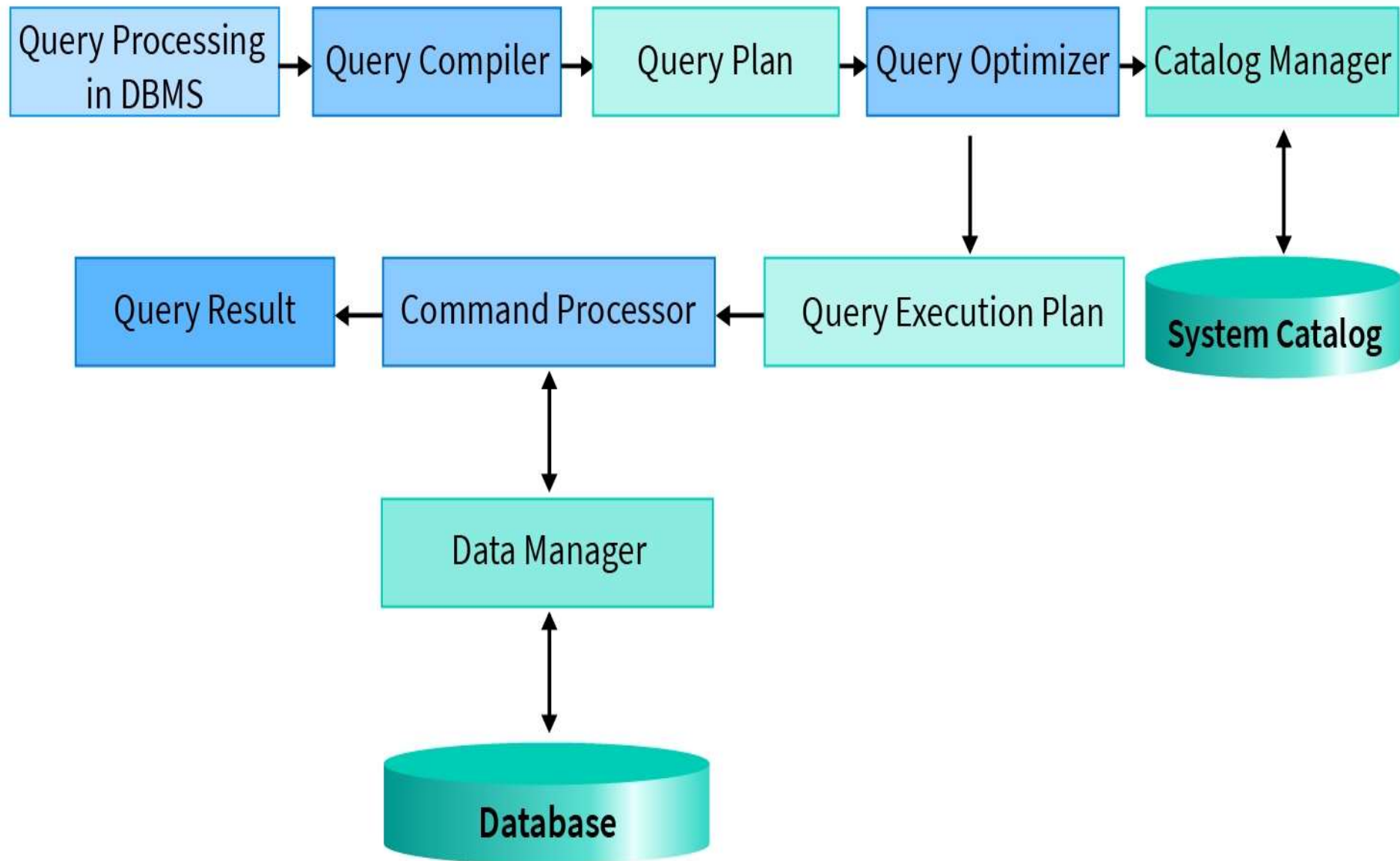
SQL

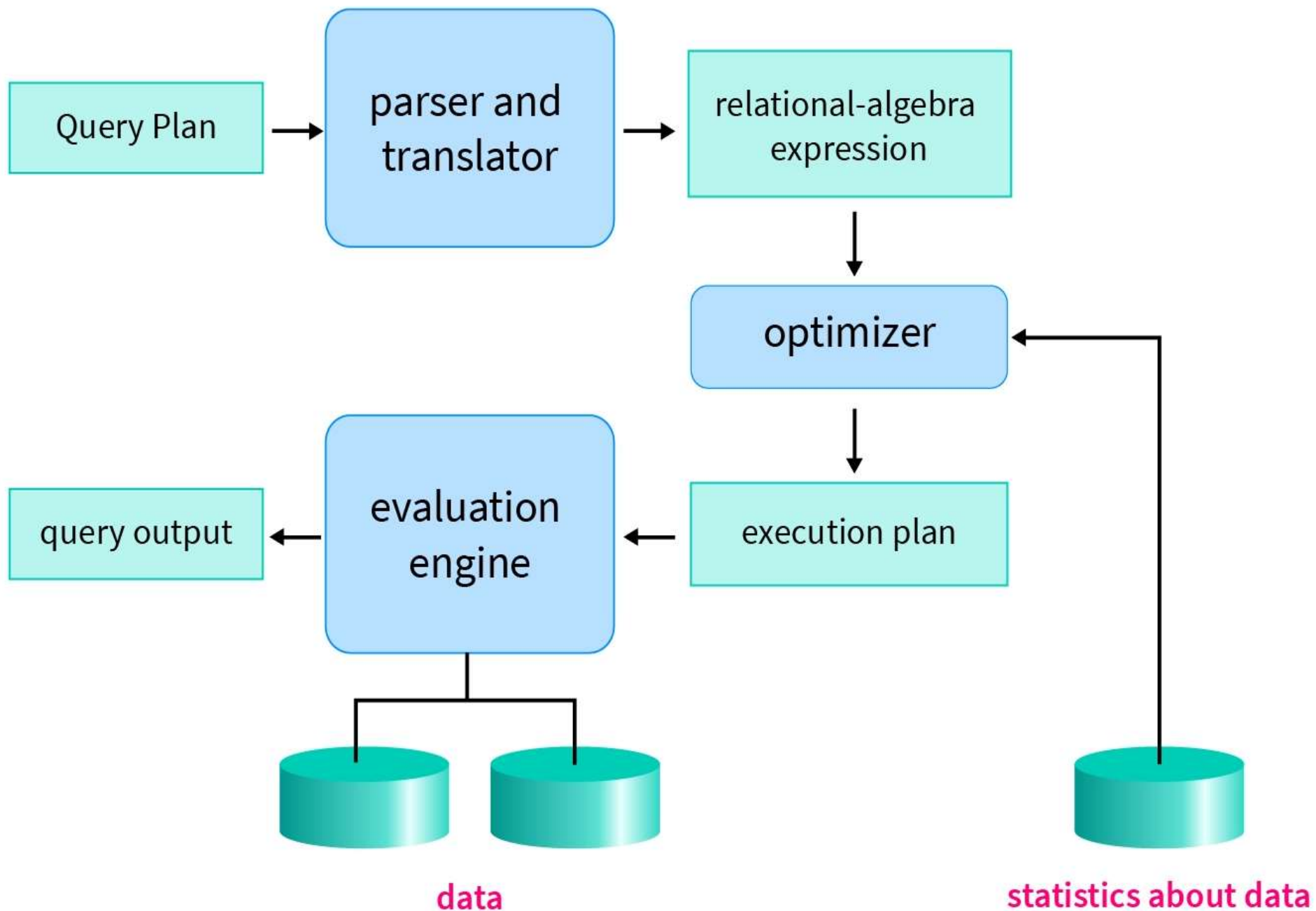
```
SELECT ProductID, ProductModelID  
FROM Product  
WHERE ProductModelID = 20 OR  
ProductModelID = 21 AND Color =  
'Red';
```

Execution modes

- n The SQL Server Database Engine can process Transact-SQL statements using two distinct processing modes:
- n Row mode execution
- n Batch mode execution







basic steps that SQL Server uses to process a single SELECT statement

- n The parser scans the SELECT statement and breaks it into logical units such as keywords, expressions, operators, and identifiers.
- n A query tree, sometimes referred to as a **sequence tree**, is built describing the logical steps needed to transform the source data into the format required by the result set.
- n The **Query Optimizer** analyzes different ways the source tables can be accessed. It then selects the series of steps that return the results fastest while using fewer resources. The query tree is updated to record this exact series of steps. The final, optimized version of the query tree is called the execution plan.
- n The relational engine starts executing the execution plan. As the steps that require data from the base tables are processed, the relational engine requests that the storage engine pass up data from the rowsets requested from the relational engine.
- n The relational engine processes the data returned from the storage engine into the format defined for the result set and returns the result set to the client.

```
SELECT
    emp_name
FROM
    employee
WHERE
    salary > 10000;
```

Π emp_name (σ salary > 10000)

- **Example:**

For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.

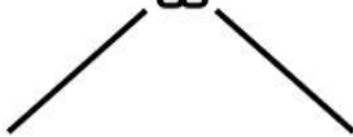
Relation algebra:

$$\pi_{\text{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}} (((\sigma_{\text{PLOCATION='STAFFORD'}}(\text{PROJECT}))$$
$$\text{DNUM=DNUMBER} (\text{DEPARTMENT})) \quad \text{MGRSSN=SSN} (\text{EMPLOYEE}))$$

SQL query:

```
Q2: SELECT P.NUMBER,P.DNUM,E.LNAME, E.ADDRESS, E.BDATE
      FROM   PROJECT AS P,DEPARTMENT AS D, EMPLOYEE AS E
      WHERE  P.DNUM=D.DNUMBER AND D.MGRSSN=E.SSN AND
            P.PLOCATION='STAFFORD';
```

Π STD_ID_STD_NAME.ADDRESS.

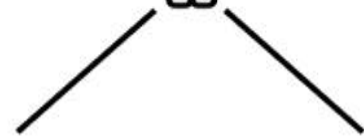


σ CLASS_NAME = 'DESIGN_01' STUDENT Table



CLASS Table

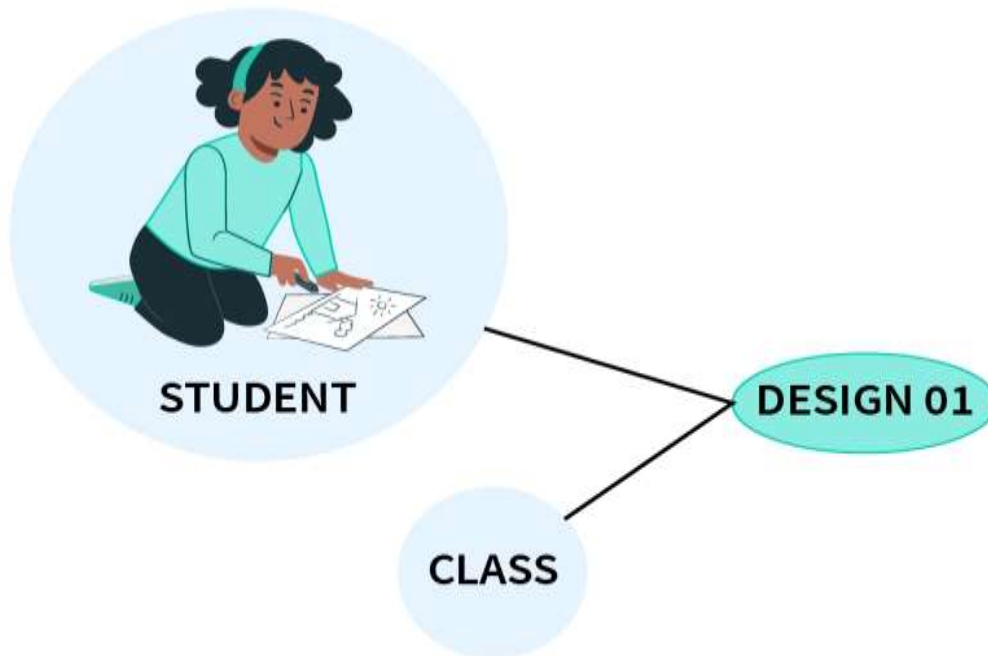
σ CLASS_NAME = 'DESIGN_01'

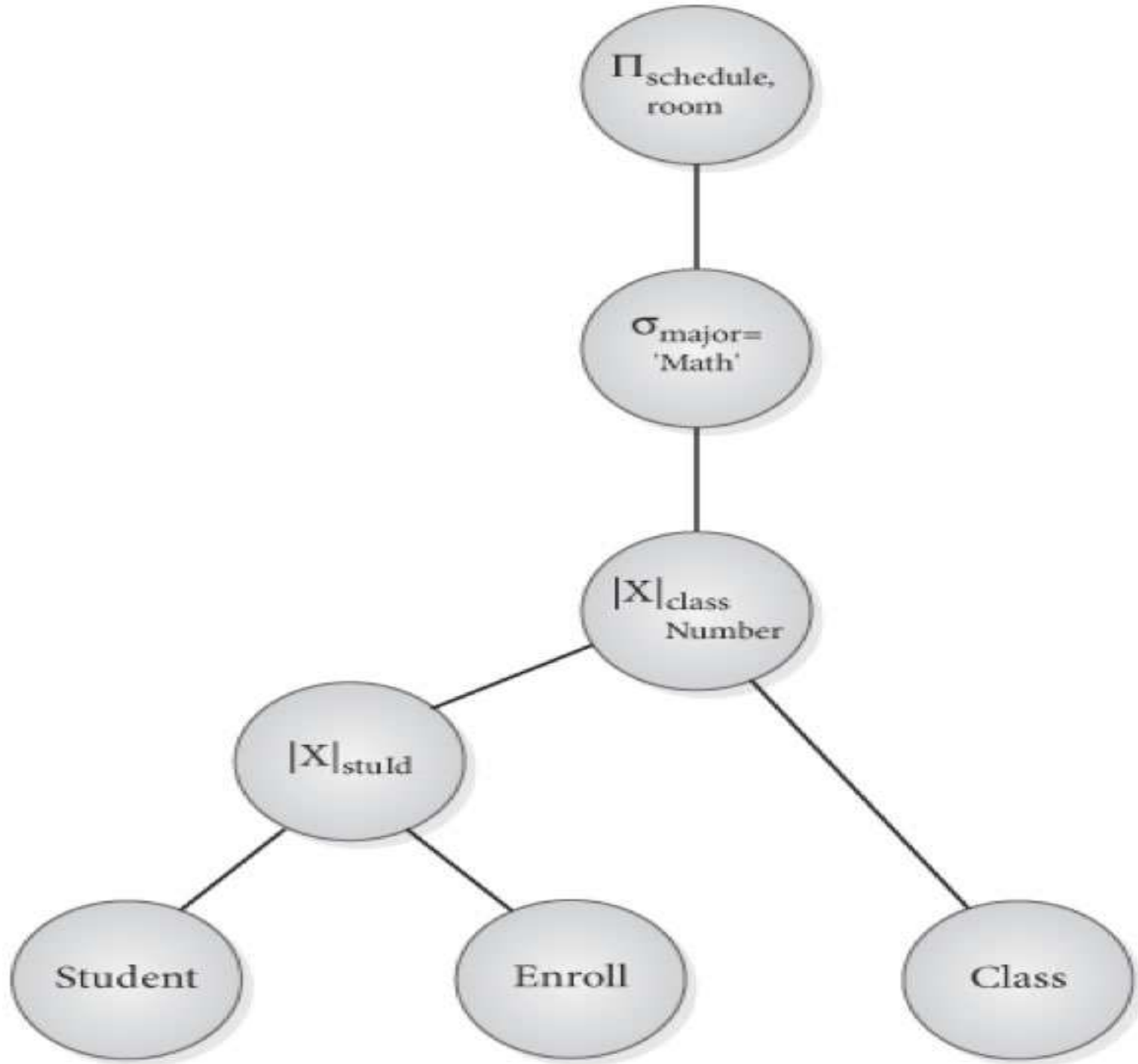


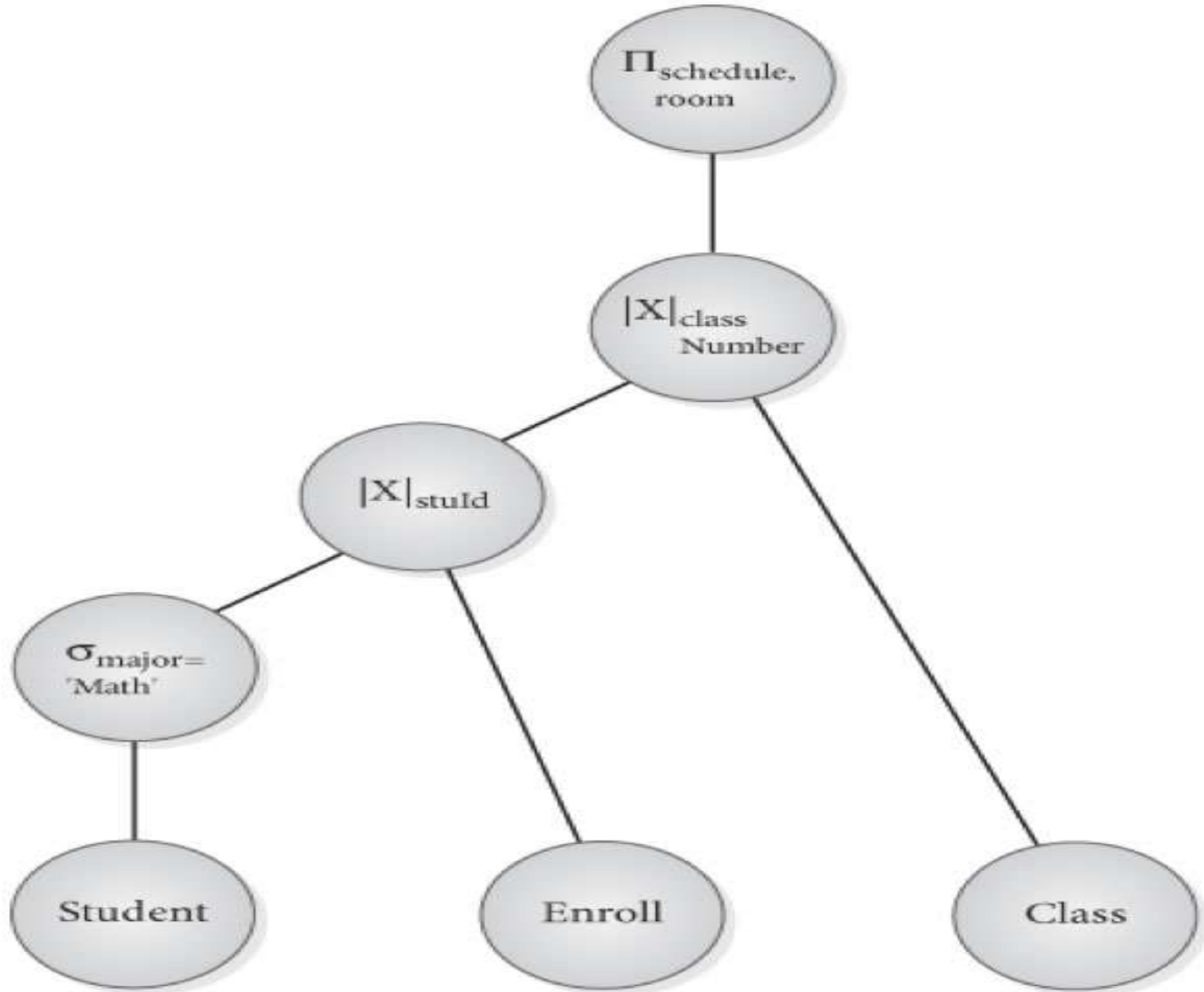
Π STD_ID_STD_NAME.ADDRESS.DOB STUDENT Table



CLASS Table







```
SELECT Name, OrderDate  
FROM Customer, Order  
WHERE Customer.Custnum =  
Order.Custnum AND  
Order."OrderDate" =  
Order."ShipDate";
```

```
n SELECT * FROM EMPLOYEE  
n WHERE SALARY < ( SELECT  
  AVERAGE(SALARY) FROM  
  EMPLOYEE ) ;
```

Translating SQL Queries into Relational Algebra

```
SELECT LNAME, FNAME  
FROM   EMPLOYEE  
WHERE  SALARY > (SELECT MAX (SALARY)  
                  FROM EMPLOYEE  
                  WHERE DNO = 5);
```

```
SELECT LNAME, FNAME  
FROM   EMPLOYEE  
WHERE  SALARY > C
```

```
SELECT MAX (SALARY)  
FROM   EMPLOYEE  
WHERE  DNO = 5
```

$\pi_{LNAME, FNAME}(\sigma_{SALARY > C}(EMPLOYEE))$

$\mathcal{F}_{MAX\ SALARY}(\sigma_{DNO=5}(EMPLOYEE))$

```
SELECT PID, STATUS FROM  
PROJECT
```

```
WHERE PID = ( SELECT FROM  
WORKS WHERE EMPID = (  
SELECT EMPID FROM EMPLOYEE  
WHERE NAME = 'ARUN  
KUMAR'));
```

Query Optimization

- n Factors weighed in by the optimizer to calculate the cost of a query evaluation plan is:
- n CPU time
- n Number of tuples to be scanned
- n Disk access time
- n number of operations

Algorithms for SELECT and JOIN Operations

Implementing the SELECT Operation:

Examples:

(OP1): $\sigma_{SSN='123456789'}(EMPLOYEE)$

(OP2): $\sigma_{DNUMBER>5}(DEPARTMENT)$

(OP3): $\sigma_{DNO=5}(EMPLOYEE)$

(OP4): $\sigma_{DNO=5 \text{ AND } SALARY>30000 \text{ AND } SEX=F}(EMPLOYEE)$

(OP5): $\sigma_{ESSN=123456789 \text{ AND } PNO=10}(WORKS_ON)$

Algorithms for SELECT and JOIN Operations

Implementing the SELECT Operation (cont.):

Search Methods for Simple Selection:

S1.Linear search (brute force): Retrieve *every record* in the file, and test whether its attribute values satisfy the selection condition.

S2.Binary search: If the selection condition involves an equality comparison on a key attribute on which the file is ordered, binary search (which is more efficient than linear search) can be used. (See OP1).

S3.Using a primary index or hash key to retrieve a single record: If the selection condition involves an equality comparison on a key attribute with a primary index (or a hash key), use the primary index (or the hash key) to retrieve the record.

Selection Using File Scan

- n Algorithm **A1** (**linear search**). Scan each file block and test all records to see whether they satisfy the selection condition.
 - | Cost estimate = b_r block transfers + 1 seek
 - ▶ b_r denotes number of blocks containing records from relation r
 - | If selection is on a key attribute, can stop on finding record
 - ▶ cost = $(b_r/2)$ block transfers + 1 seek
 - | Linear search can be applied regardless of
 - ▶ selection condition or
 - ▶ ordering of records in the file, or
 - ▶ availability of indices
- n Note: binary search generally does not make sense since data is not stored consecutively
 - | except when there is an index available,
 - | and binary search requires more seeks than index search

Selections Using Indices

- n **Index scan** – search algorithms that use an index
 - | selection condition must be on search-key of index.
- n **Algorithm A2 (clustering index, equality on key).**
Retrieve a single record that satisfies the corresponding equality condition
 - | $Cost = (h_i + 1) * (t_T + t_S)$
- n **Algorithm A3 (clustering index, equality on nonkey)**
Retrieve multiple records.
 - | Records will be on consecutive blocks
 - ▶ Let b = number of blocks containing matching records
 - | $Cost = h_i * (t_T + t_S) + t_S + t_T * b$

Selections Involving Comparisons

- n Can implement selections of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$ by using
 - | a linear file scan,
 - | or by using indices in the following ways:
- n **Algorithm A5 (clustering index, comparison).** (Relation is sorted on A)
 - ▶ For $\sigma_{A \geq v}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - ▶ For $\sigma_{A \leq v}(r)$ just scan relation sequentially till first tuple $> v$; do not use index
- n **Algorithm A6 (clustering index, comparison).**
 - ▶ For $\sigma_{A \geq v}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
 - ▶ For $\sigma_{A \leq v}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
 - ▶ In either case, retrieve records that are pointed to
 - ▶ requires an I/O per record; Linear file scan may be cheaper!

Implementation of Complex Selections

- n **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- n **Algorithm A7 (conjunctive selection using one index).**
 - | Select a combination of θ_i and algorithms A1 through A7 that results in the least cost for $\sigma_{\theta_i}(r)$.
 - | Test other conditions on tuple after fetching it into memory buffer.
- n **Algorithm A8 (conjunctive selection using composite index).**
 - | Use appropriate composite (multiple-key) index if available.
- n **Algorithm A9 (conjunctive selection by intersection of identifiers).**
 - | Requires indices with record pointers.
 - | Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.
 - | Then fetch records from file
 - | If some conditions do not have appropriate indices, apply test in memory.

Algorithms for Complex Selections

- n **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.
- n **Algorithm A10 (disjunctive selection by union of identifiers).**
 - | Applicable if *all* conditions have available indices.
 - ▶ Otherwise use linear scan.
 - | Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
 - | Then fetch records from file
- n **Negation:** $\sigma_{\neg\theta}(r)$
 - | Use linear scan on file
 - | If very few records satisfy $\neg\theta$, and an index is applicable to θ
 - ▶ Find satisfying records using index and fetch from file

Implementing the JOIN Operation:

Join (EQUIJOIN, NATURAL JOIN)

two-way join: a join on two files

e.g. $R \bowtie_{A=B} S$

multi-way joins: joins involving more than two files.

e.g. $R \bowtie_{A=B} S \bowtie_{C=D} T$

Examples

(OP6): EMPLOYEE $\bowtie_{DNO=DNUMBER}$ DEPARTMENT

(OP7): DEPARTMENT $\bowtie_{MGRSSN=SSN}$ EMPLOYEE

Join Operation

- n Several different algorithms to implement joins
 - | Nested-loop join
 - | Block nested-loop join
 - | Indexed nested-loop join
 - | Merge-join
 - | Hash-join
- n Choice based on cost estimate
- n Examples use the following information
 - | Number of records of *student*: 5,000 *takes*: 10,000
 - | Number of blocks of *student*: 100 *takes*: 400

Nested-Loop Join

- n To compute the theta join $r \bowtie_{\theta} s$
 - for each** tuple t_r **in** r **do begin**
 - for each** tuple t_s **in** s **do begin**
 - test pair (t_r, t_s) to see if they satisfy the join condition θ
 - if they do, add $t_r \cdot t_s$ to the result.
 - end**
- end**
- n r is called the **outer relation** and s the **inner relation** of the join.

Block Nested-Loop Join

- Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block  $B_r$  of  $r$  do begin  
  for each block  $B_s$  of  $s$  do begin  
    for each tuple  $t_r$  in  $B_r$  do begin  
      for each tuple  $t_s$  in  $B_s$  do begin  
        Check if  $(t_r, t_s)$  satisfy the join condition  
        if they do, add  $t_r \cdot t_s$  to the result.  
      end  
    end  
  end  
end
```

Indexed Nested-Loop Join

- n Index lookups can replace file scans if
 - | join is an equi-join or **natural join** and
 - | an index is available on the inner relation's join attribute
 - ▶ Can construct an index just to compute a join.
- n For each tuple t_r in the outer relation r , use the index to look up tuples in s that satisfy the join condition with tuple t_r .
- n Worst case: buffer has space for only one page of r , and, for each tuple in r , we perform an index lookup on s .
- n Cost of the join: $b_r (t_T + t_S) + n_r * c$
 - | Where c is the cost of traversing index and fetching all matching s tuples for one tuple of r
 - | c can be estimated as cost of a single selection on s using the join condition.
- n If indices are available on join attributes of both r and s , use the relation with fewer tuples as the outer relation.

Merge-Join

1. Sort both relations on their join attribute (if not already sorted on the join attributes).
2. Merge the sorted relations to join them
 1. Join step is similar to the merge stage of the sort-merge algorithm.
 2. Main difference is handling of duplicate values in join attribute —

every pair with same value on join attribute must be matched

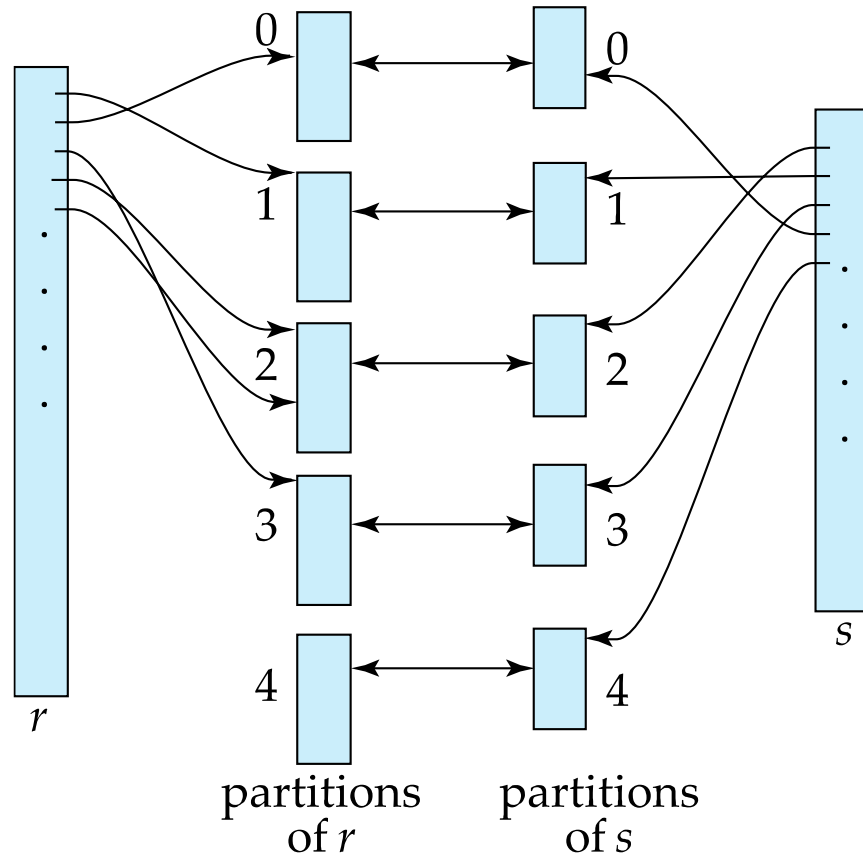
3. Detailed algorithm in book

pr		ps	
a_1	a_2	a_1	a_3
a	3	a	A
b	1	b	G
d	8	c	L
d	13	d	N
f	7	m	B
m	5		
q	6		
r		s	

Hash-Join

- n Applicable for equi-joins and natural joins.
- n A hash function h is used to partition tuples of both relations
- n h maps *JoinAttrs* values to $\{0, 1, \dots, n\}$, where *JoinAttrs* denotes the common attributes of r and s used in the natural join.
 - | $r_0, r_1, \dots, r_n$ denote partitions of r tuples
 - ▶ Each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[\text{JoinAttrs}])$.
 - | $r_0, r_1, \dots, r_n$ denotes partitions of s tuples
 - ▶ Each tuple $t_s \in s$ is put in partition s_i , where $i = h(t_s[\text{JoinAttrs}])$.
- n Note: In book, Figure 12.10 r_i is denoted as H_{ri} , s_i is denoted as H_{si} and n is denoted as n_h

Hash-Join (Cont.)



Hash-Join Algorithm

The hash-join of r and s is computed as follows.

1. Partition the relation s using hashing function h . When partitioning a relation, one block of memory is reserved as the output buffer for each partition.
2. Partition r similarly.
3. For each i :
 - (a) Load s_i into memory and build an in-memory hash index on it using the join attribute. This hash index uses a different hash function than the earlier one h .
 - (b) Read the tuples in r_i from the disk one by one. For each tuple t_r locate each matching tuple t_s in s_i using the in-memory hash index. Output the concatenation of their attributes.

Complex Joins

■ Join with a conjunctive condition:

$$r \bowtie_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n} s$$

- Either use nested loops/block nested loops, or
- Compute the result of one of the simpler joins $r \bowtie_{\theta_i} s$
 - ▶ final result comprises those tuples in the intermediate result that satisfy the remaining conditions

$$\theta_1 \wedge \dots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \dots \wedge \theta_n$$

■ Join with a disjunctive condition

$$r \bowtie_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n} s$$

- Either use nested loops/block nested loops, or
- Compute as the union of the records in individual joins $r \bowtie_{\theta_i} s$:

$$(r \bowtie_{\theta_1} s) \cup (r \bowtie_{\theta_2} s) \cup \dots \cup (r \bowtie_{\theta_n} s)$$

Other Operations

- n **Duplicate elimination** can be implemented via hashing or sorting.
 - | On sorting duplicates will come adjacent to each other, and all but one set of duplicates can be deleted.
 - | *Optimization:* duplicates can be deleted during run generation as well as at intermediate merge steps in external sort-merge.
 - | Hashing is similar – duplicates will come into the same bucket.
- n **Projection:**
 - | perform projection on each tuple
 - | followed by duplicate elimination.

Other Operations : Aggregation

- n **Aggregation** can be implemented in a manner similar to duplicate elimination.
 - | **Sorting** or **hashing** can be used to bring tuples in the same group together, and then the aggregate functions can be applied on each group.
 - | Optimization: **partial aggregation**
 - ▶ combine tuples in the same group during run generation and intermediate merges, by computing partial aggregate values
 - ▶ For count, min, max, sum: keep aggregate values on tuples found so far in the group.
 - When combining partial aggregate for count, add up the partial aggregates
 - ▶ For avg, keep sum and count, and divide sum by count at the end

The following set of $n = 16$ numbers is to be sorted:

19, 11, 03, 05, 02, 07, 31, 17, 13, 29, 23, 41, 37, 51, 47, 43
--

The system's main memory is capable of storing $p = 4$ numbers. It uses no buffers.

To sort the file internally it has to be divided into 4 subfiles, that fit into main memory:

19, 11, 03, 05	02, 07, 31, 17	13, 29, 23, 41	37, 51, 47, 43
----------------	----------------	----------------	----------------

These 4 subfiles have to be sorted:

19, 11, 03, 05	internal sorting	03, 05, 11, 19
02, 07, 31, 17	internal sorting	02, 07, 17, 31
13, 29, 23, 41	internal sorting	13, 23, 29, 41
37, 51, 47, 43	internal sorting	37, 43, 47, 51

Other Operations : Set Operations

- **Set operations** ($\cup$, $\cap$ and $-$): can either use variant of merge-join after sorting, or variant of hash-join.
- E.g., Set operations using hashing:
 1. Partition both relations using the same hash function
 2. Process each partition i as follows.
 1. Using a different hashing function, build an in-memory hash index on r_i .
 2. Process s_i as follows
 - $r \cup s$:
 1. Add tuples in s_i to the hash index if they are not already in it.
 2. At end of s_i add the tuples in the hash index to the result.

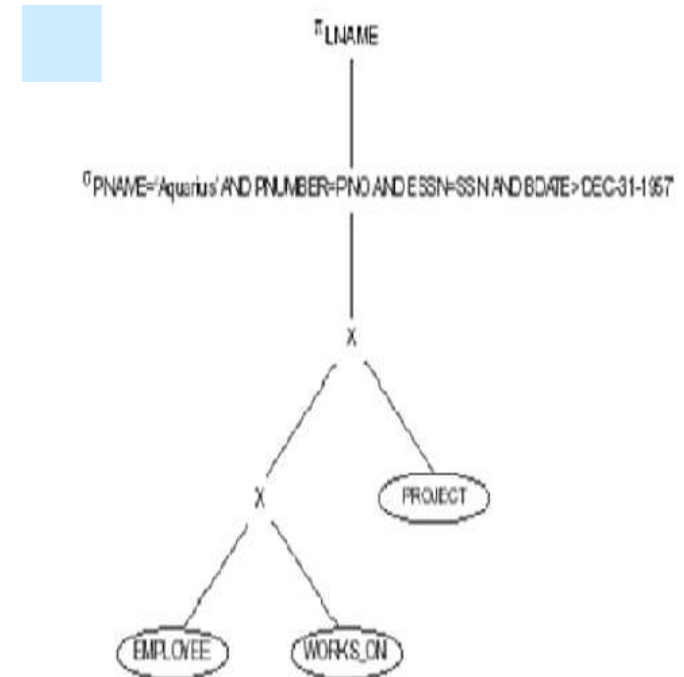
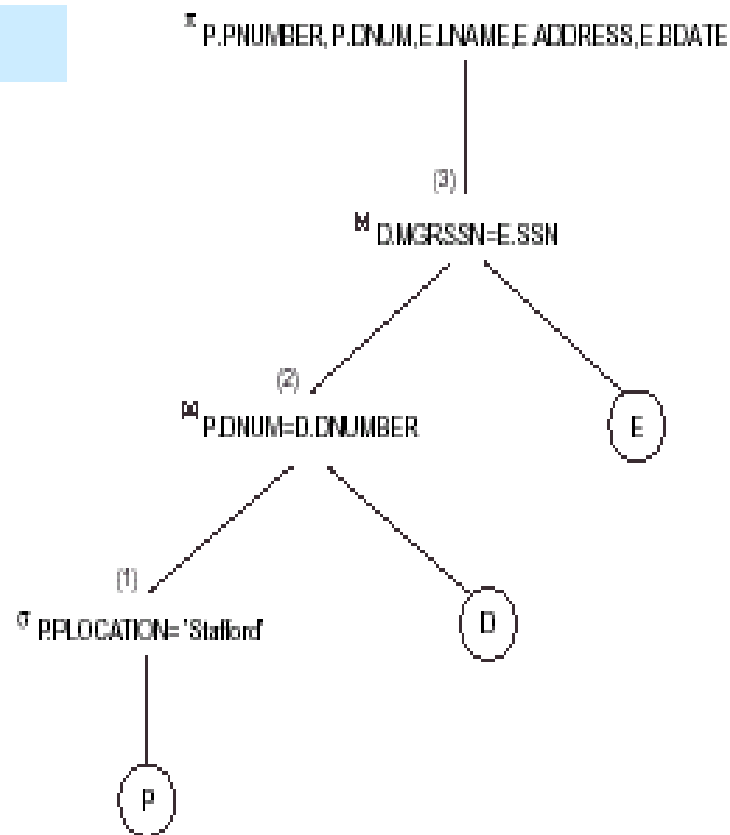
Other Operations : Set Operations

- E.g., Set operations using hashing:
 1. as before partition r and s ,
 2. as before, process each partition i as follows
 1. build a hash index on r_i
 2. Process s_i as follows
 - $r \cap s$:
 1. output tuples in s_i to the result if they are already there in the hash index
 - $r - s$:
 1. for each tuple in s_i , if it is there in the hash index, delete it from the index.
 2. At end of s_i add remaining tuples in the hash index to the result.

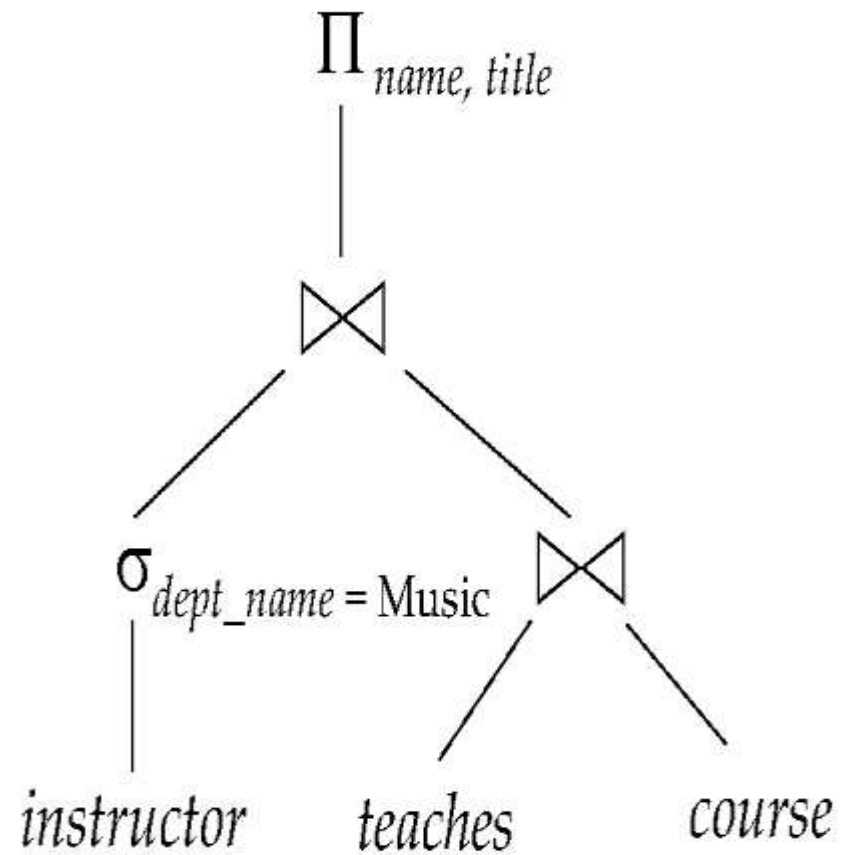
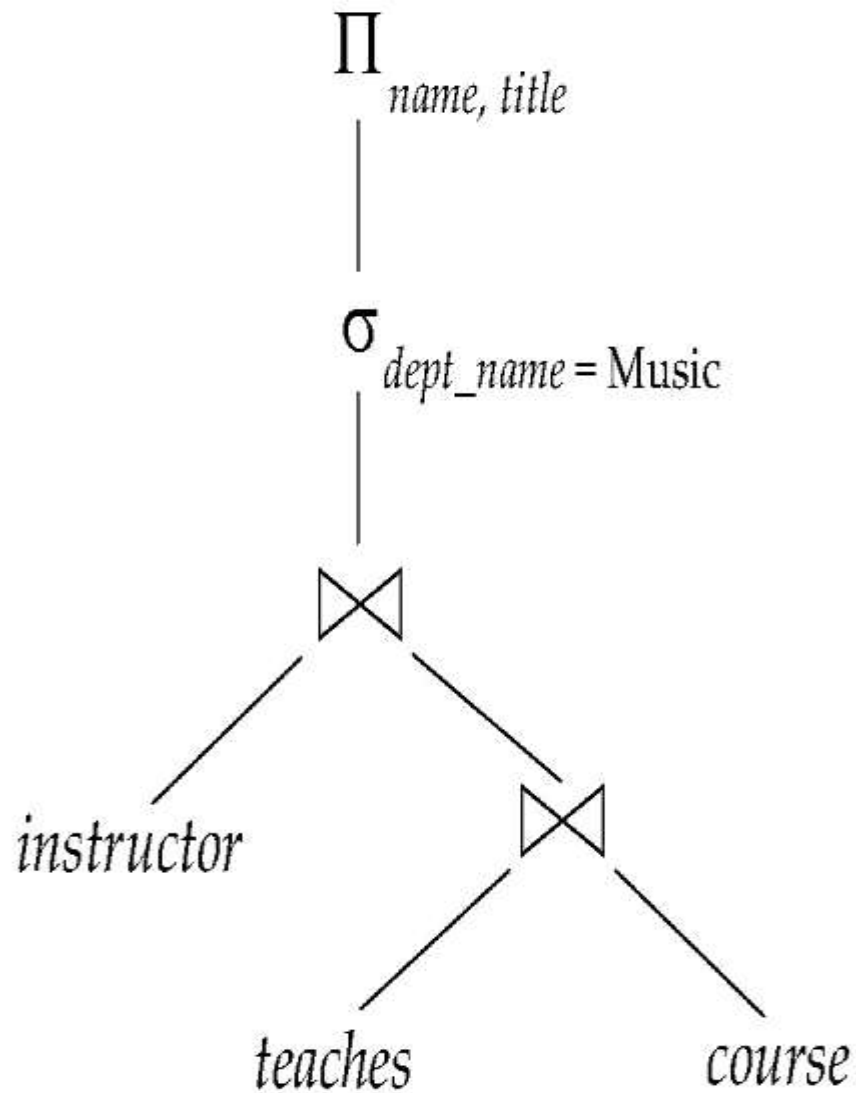
Other Operations : Outer Join

- n **Outer join** can be computed either as
 - | A join followed by addition of null-padded non-participating tuples.
 - | by modifying the join algorithms.
- n Modifying merge join to compute $r \bowtie s$
 - | In $r \bowtie s$, non participating tuples are those in $r - \Pi_R(r \bowtie s)$
 - | Modify merge-join to compute $r \bowtie s$:
 - ▶ During merging, for every tuple t_r from r that do not match any tuple in s , output t_r padded with nulls.
 - | Right outer-join and full outer-join can be computed similarly.

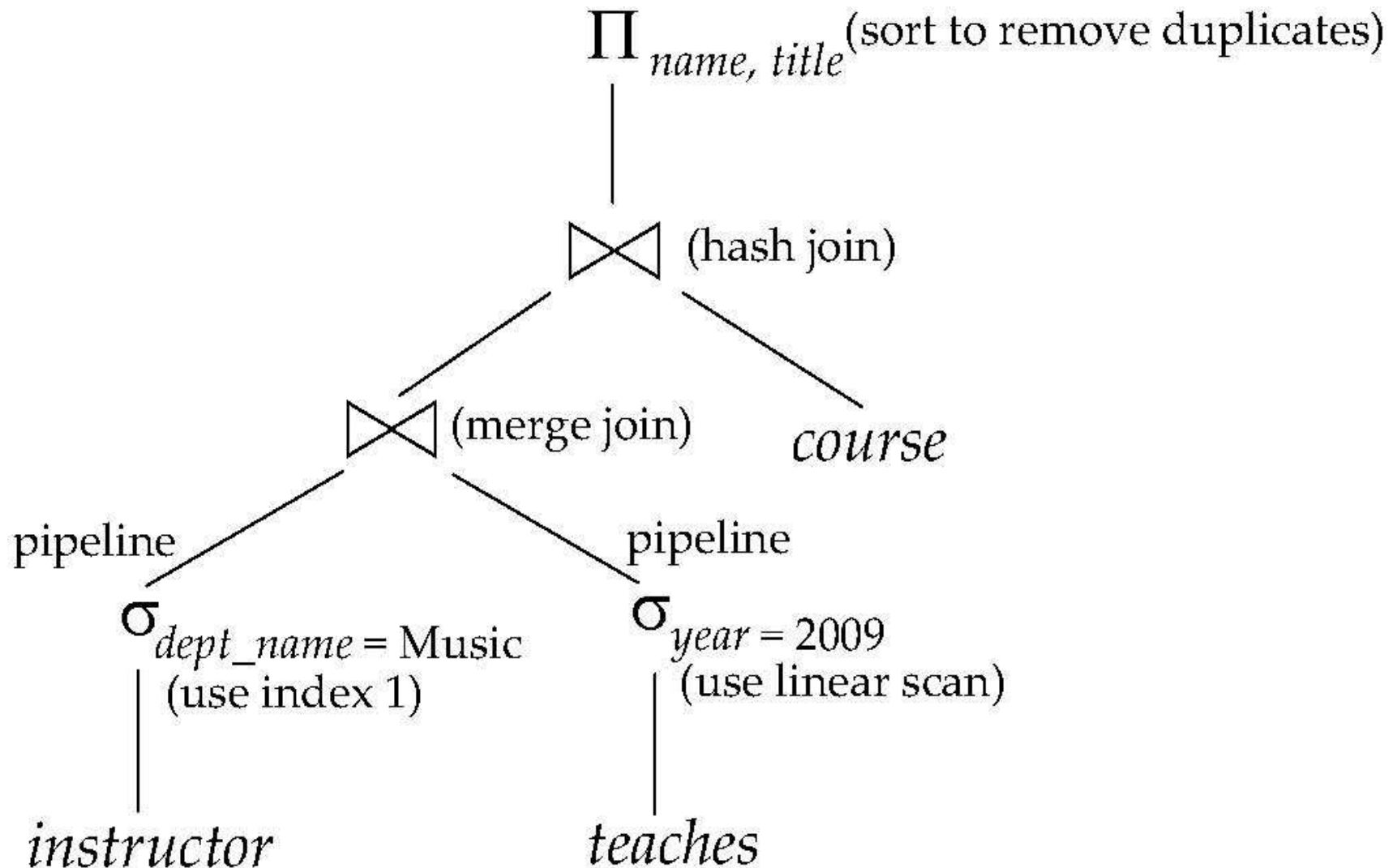
The same query could correspond to many different relational algebra expressions — and hence many different query trees.



Equivalent expressions



- n An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



Transformation of Relational Expressions

- n Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - | Note: order of tuples is irrelevant
 - | we don't care if they generate different results on databases that violate integrity constraints
- n In SQL, inputs and outputs are multisets of tuples
 - | Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- n An **equivalence rule** says that expressions of two forms are equivalent
 - | Can replace expression of first form by second, or vice

Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$s_{q_1 \cup q_2}(E) = s_{q_1}(s_{q_2}(E))$$

2. Selection operations are commutative.

$$s_{q_1}(s_{q_2}(E)) = s_{q_2}(s_{q_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

- a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

- b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6. (a) Natural join operations are associative:

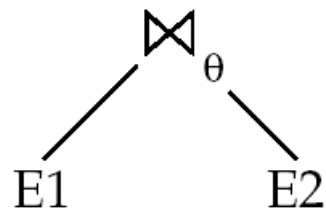
$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- (b) Theta joins are associative in the following manner:

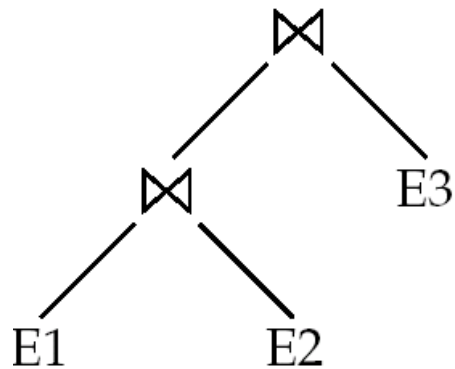
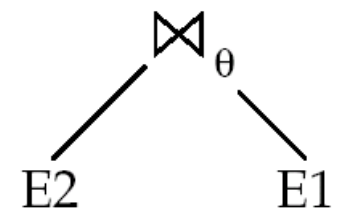
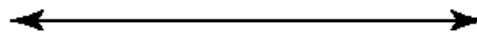
$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

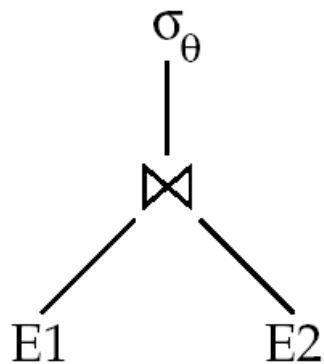
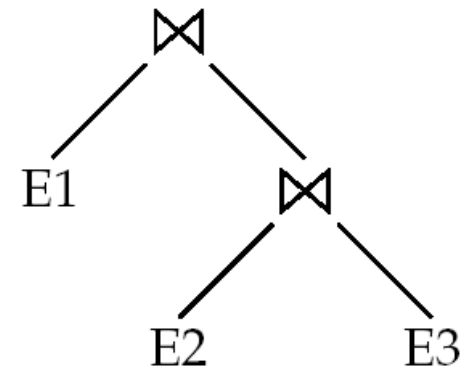
Pictorial Depiction of Equivalence Rules



Rule 5



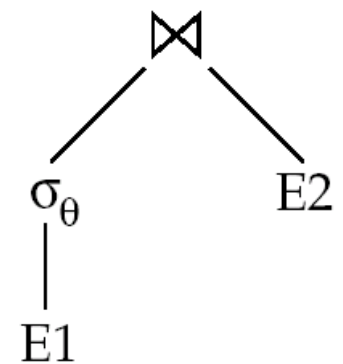
Rule 6a



Rule 7a



If θ only has
attributes from $E1$



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

(b) Consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

□ (set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta} (E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also:
$$\sigma_{\theta} (E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

(CONT...)

Query Tree Optimization

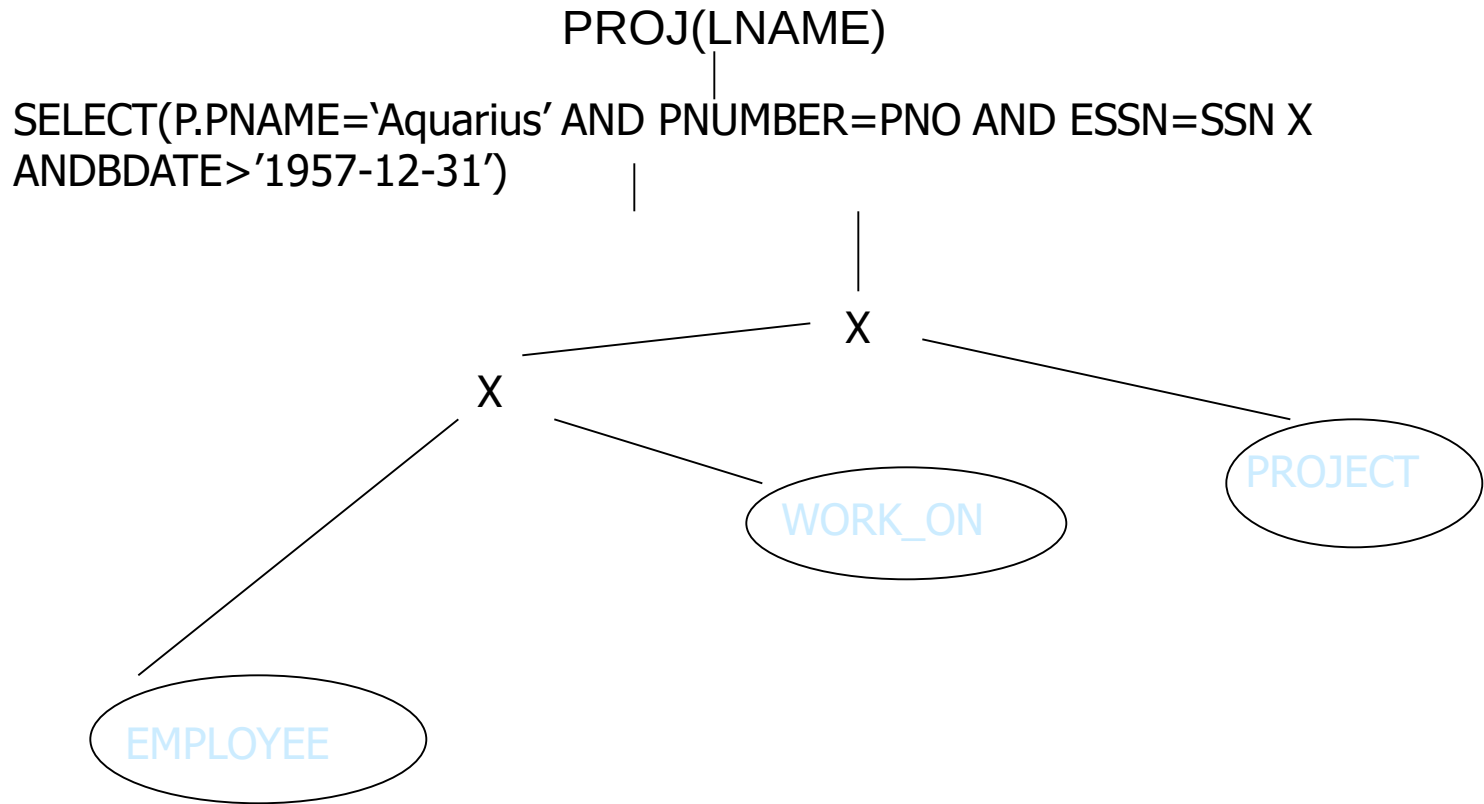


FIG1 : INITIAL QUERY TREE

(CONT...)

Query Tree Optimization

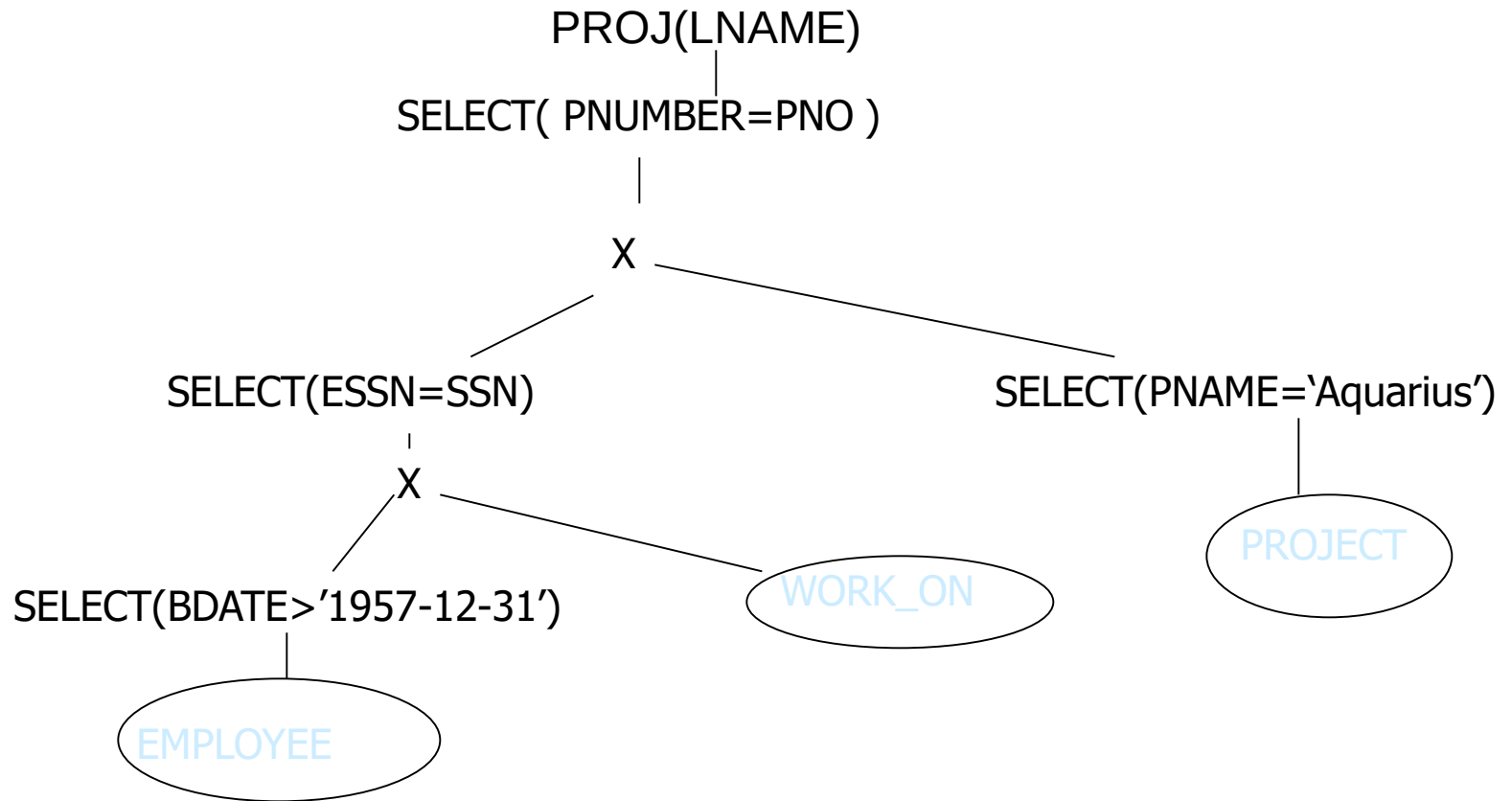


FIG2 : MOVE SELECT DOWN THE TREE USING CASCADE & COMMUTATIVITY RULE OF SELECT OPERATION

(CONT...)

Query Tree Optimization

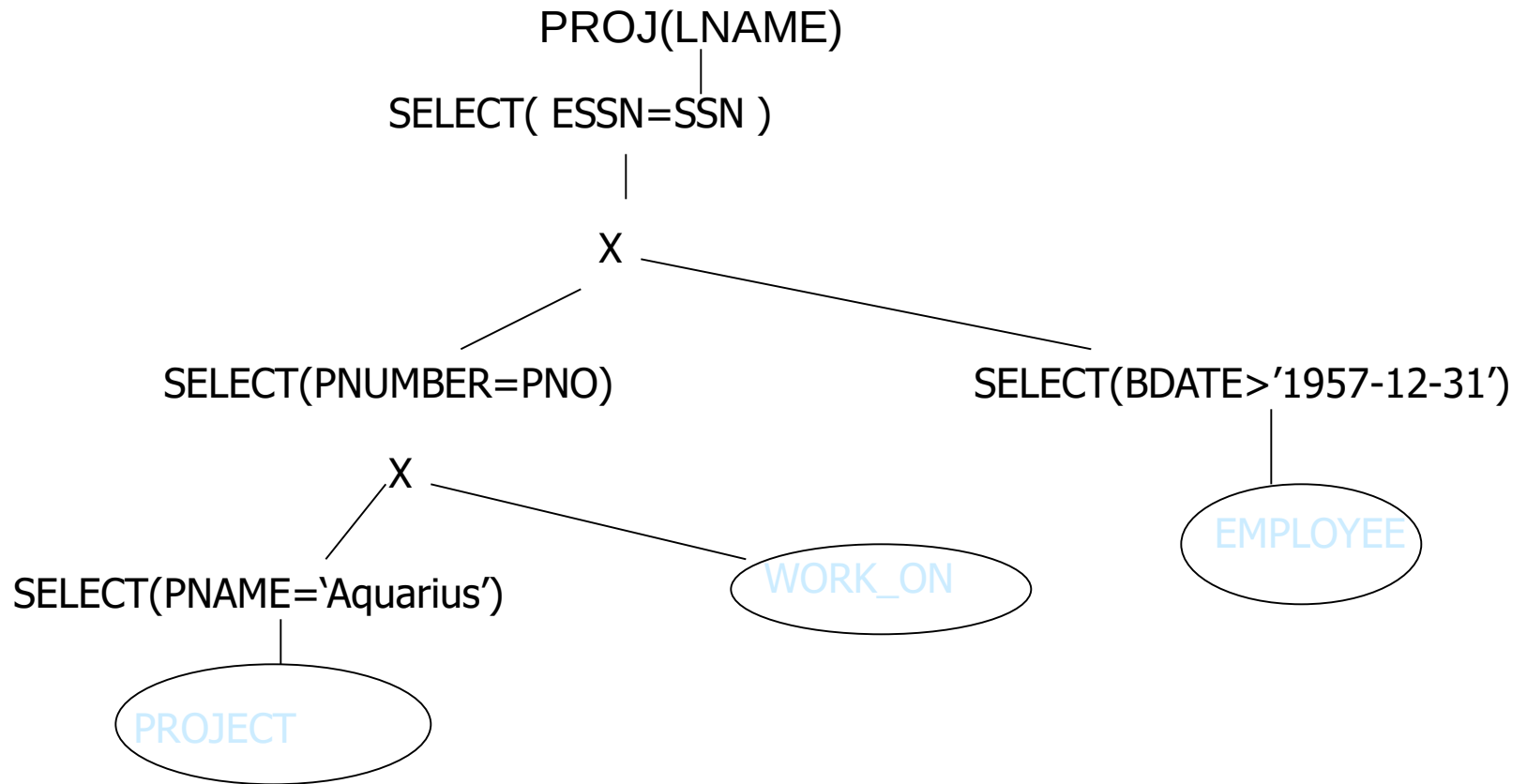


FIG3 : REARRANGE OF LEAF NODES, USING COMMUTATIVITY & ASSOCIATIVITY OF BINARY OPERATIONS.

(CONT...)

Query Tree Optimization

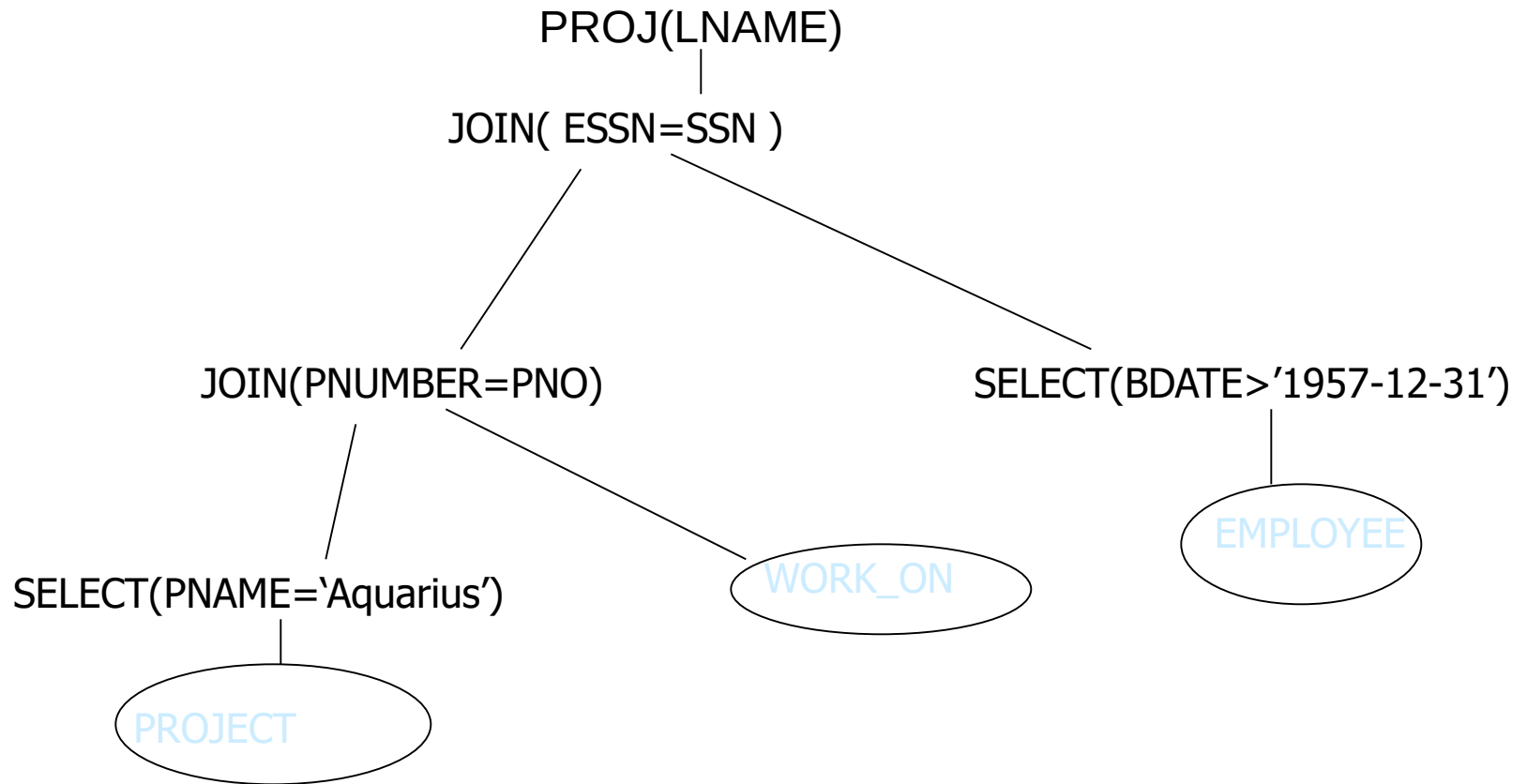


FIG4 : CONVERTING SELECT & CARTESIAN PRODUCT INTO JOIN

(CONT...)

Query Tree Optimization

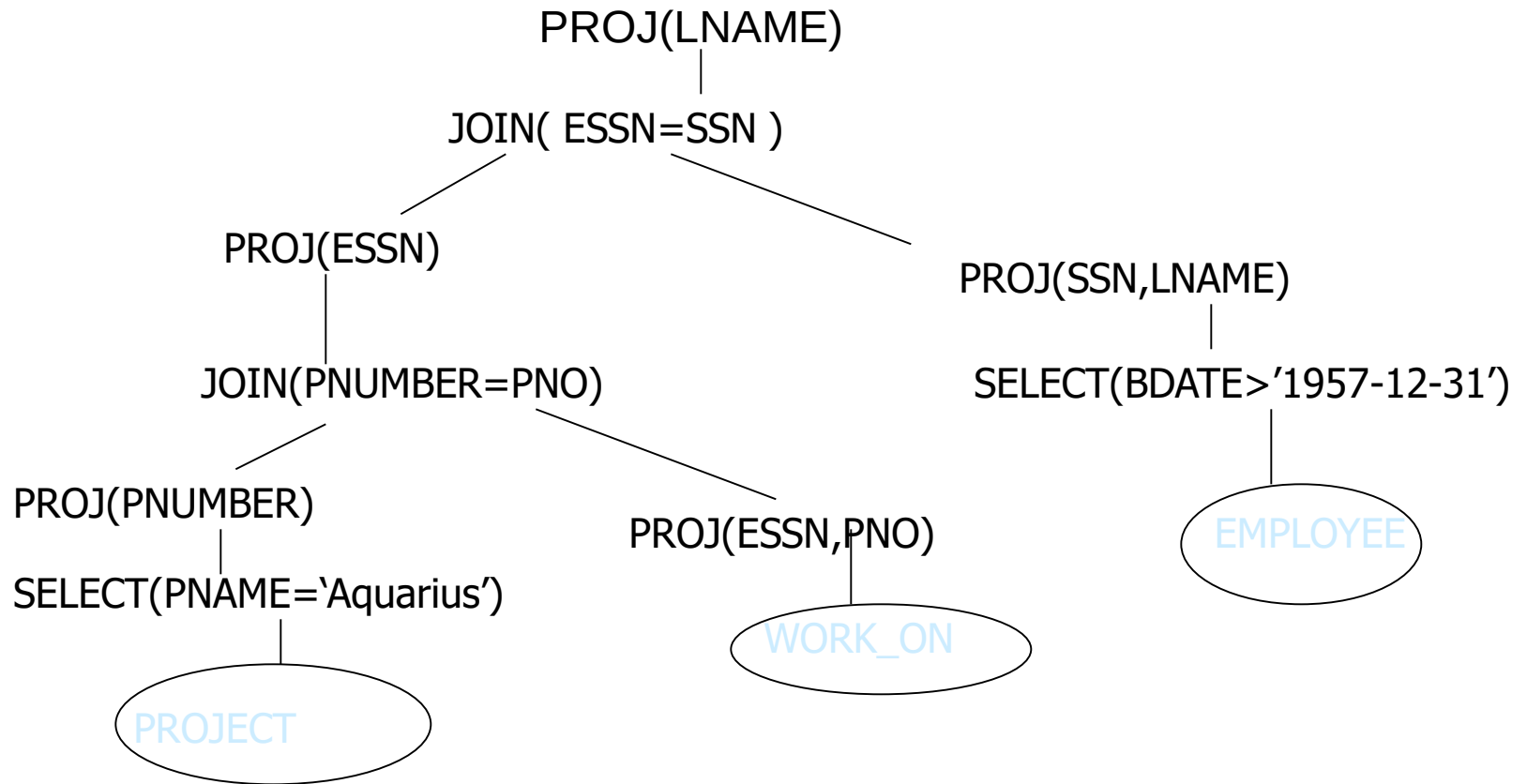


FIG4 : BREAK-MOVE OF PROJECT USING CASCADE & COMMUTING RULES OF PROJECT OPERATIONS.

Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach

- $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$

- Transformation using rule 7a.

- $\Pi_{name, title}((\sigma_{dept_name = \text{"Music"}}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$

- Performing the selection as early as possible reduces the size of the relation to be joined.

Example with Multiple Transformations

- Query: Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught

- $\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge year = 2009} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title} (course))))$

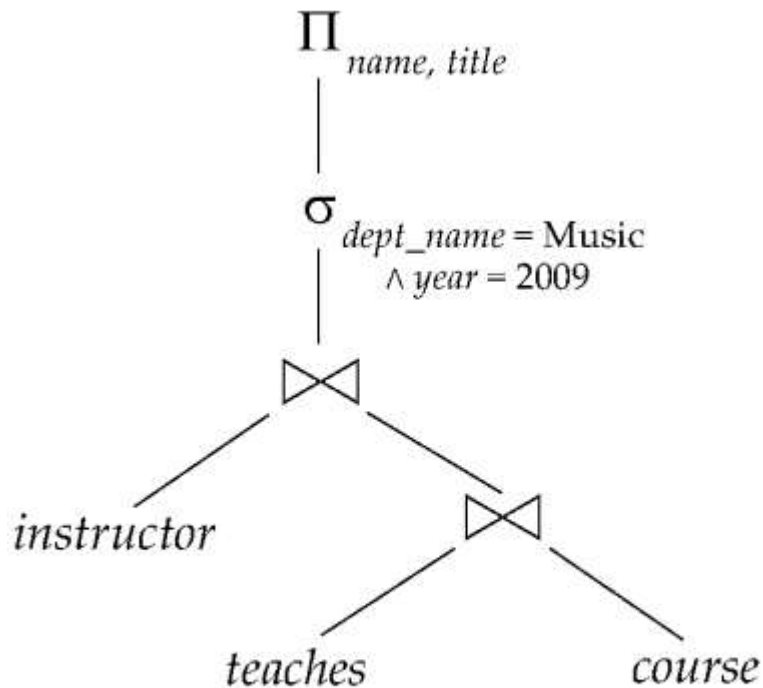
- Transformation using join associatively (Rule 6a):

- $\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge year = 2009} ((instructor \bowtie teaches) \bowtie \Pi_{course_id, title} (course)))$

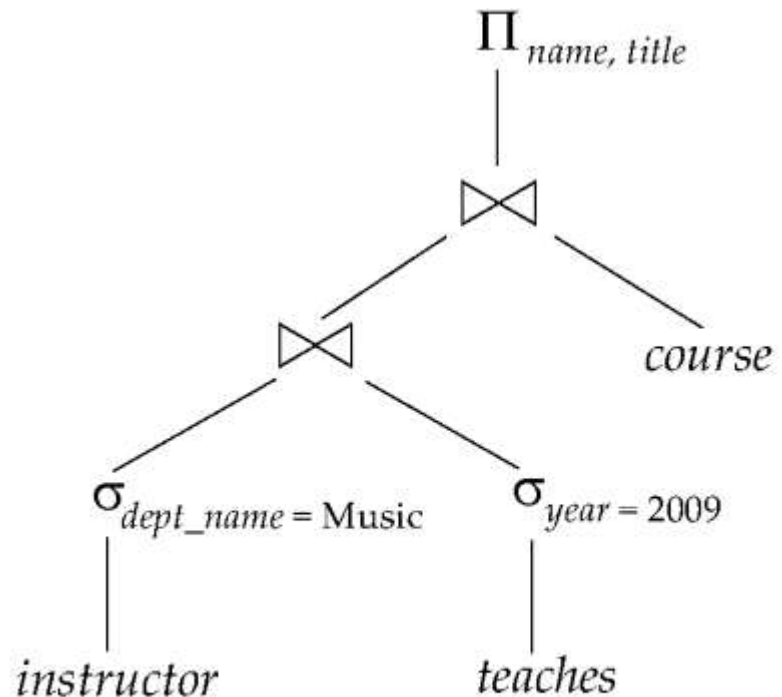
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

$$\sigma_{dept_name = \text{“Music”}} (instructor) \bowtie \sigma_{year = 2009} (teaches)$$

Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations

Transformation Example: Pushing Projections

- Consider: $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$

- When we compute

$$(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches)$$

we obtain a relation whose schema is:

$(ID, name, dept_name, salary, course_id, sec_id, semester, year)$

- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{"Music"}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$$

- Performing the projection as early as possible reduces the size of the relation to be joined.

Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity)

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.

Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

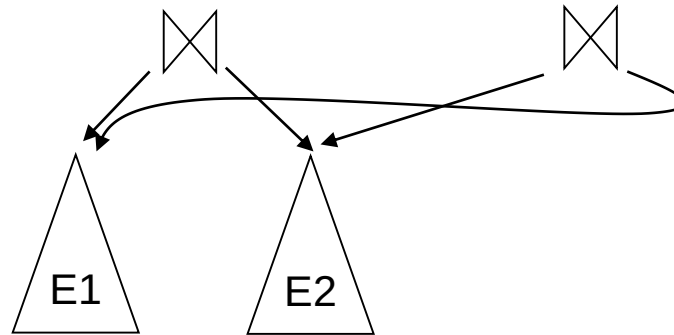
- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

$$\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches$$

first.

Implementing Transformation Based Optimization

- Space requirements reduced by sharing common sub-expressions:
 - when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers
 - ▶ E.g. when applying join commutativity



- Same sub-expression may get generated multiple times
 - ▶ Detect duplicate sub-expressions and share one copy
- Time requirements are reduced by not generating all expressions
 - Dynamic programming
 - ▶ We will study only the special case of dynamic programming for join order optimization

Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
 - ▶ merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
 - ▶ nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.

Cost-Based Optimization

- Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \dots r_n$.
- There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \dots, r_n\}$ is computed only once and stored for future use.

Dynamic Programming in Optimization

- To find best join tree for a set of n relations:
 - To find best plan for a set S of n relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan. Choose the cheapest of the $2^n - 2$ alternatives.
 - Base case for recursion: single relation access plan
 - ▶ Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it
 - ▶ Dynamic programming

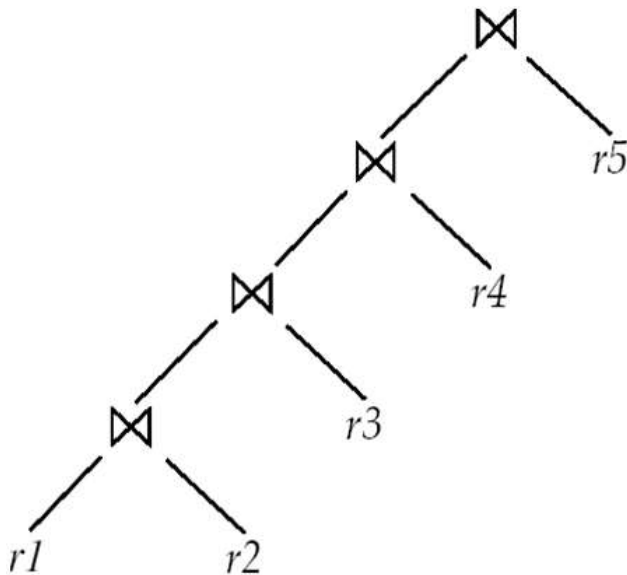
Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S /* Using selections on S and indices on S */
  else for each non-empty subset S1 of S such that S1  $\neq$  S
    P1= findbestplan(S1)
    P2= findbestplan(S - S1)
    A = best algorithm for joining results of P1 and P2
    cost = P1.cost + P2.cost + cost of A
    if cost < bestplan[S].cost
      bestplan[S].cost = cost
      bestplan[S].plan = “execute P1.plan; execute P2.plan;
                          join results of P1 and P2 using A”
  return bestplan[S]
```

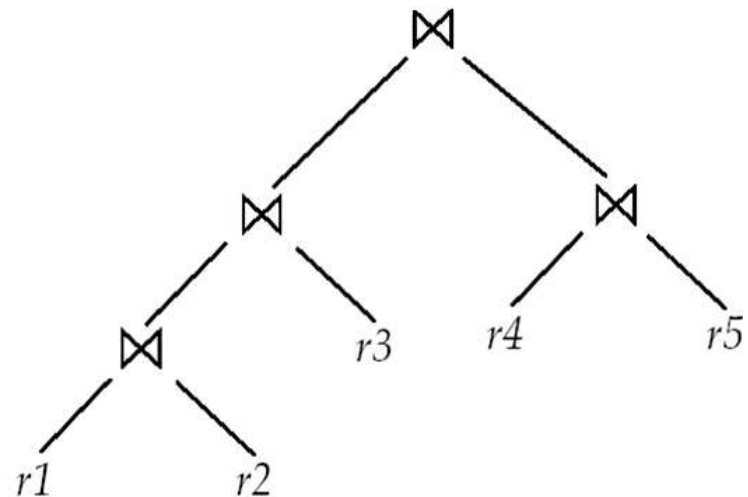
* Some modifications to allow indexed nested loops joins on relations that have selections (see book)

Left Deep Join Trees

- In **left-deep join trees**, the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree

Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
 - With $n = 10$, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations:
 - Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.
 - Modify optimization algorithm:
 - ▶ Replace “**for each** non-empty subset $S1$ of S such that $S1 \neq S$ ”
 - ▶ By: **for each** relation r in S
let $S1 = S - r$.
- If only left-deep trees are considered, time complexity of finding best join order is $O(n 2^n)$
 - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n , generally < 10)

Interesting Sort Orders

- Consider the expression $(r_1 \bowtie r_2) \bowtie r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could be useful for a later operation
 - Using merge-join to compute $r_1 \bowtie r_2$ may be costlier than hash join but generates result sorted on A
 - Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
 - Sort order may also be useful for order by and for grouping
- Not sufficient to find the best join order for each subset of the set of n given relations
 - must find the best join order for each subset, **for each interesting sort order**
 - Simple extension of earlier dynamic programming algorithms
 - Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly

Cost Based Optimization with Equivalence Rules

- **Physical equivalence rules** allow logical query plan to be converted to physical query plan specifying what algorithms are used for each operation.
- Efficient optimizer based on equivalent rules depends on
 - A space efficient representation of expressions which avoids making multiple copies of subexpressions
 - Efficient techniques for detecting duplicate derivations of expressions
 - A form of dynamic programming based on **memoization**, which stores the best plan for a subexpression the first time it is optimized, and reuses it on repeated optimization calls on same subexpression
 - Cost-based pruning techniques that avoid generating all plans
- Pioneered by the Volcano project and implemented in the SQL Server optimizer

Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples)
 - Perform projection early (reduces the number of attributes)
 - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
 - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Optimizing Nested Subqueries**

- Nested query example:
select *name*
from *instructor*
where exists (**select** *
 from *teaches*
 where *instructor.ID = teaches.ID and teaches.year = 2007*)
- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
 - Such evaluation is called **correlated evaluation**
 - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery

Optimizing Nested Subqueries (Cont.)

- Correlated evaluation may be quite inefficient since
 - a large number of calls may be made to the nested query
 - there may be unnecessary random I/O as a result
- SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques
- E.g.: earlier nested query can be rewritten as

```
select name
from instructor, teaches
where instructor.ID = teaches.ID and teaches.year = 2007
```

 - Note: the two queries generate different numbers of duplicates (why?)
 - ▶ teaches can have duplicate IDs
 - ▶ Can be modified to handle duplicates correctly as we will see
- In general, it is not possible/straightforward to move the entire nested subquery from clause into the outer level query from clause
 - A temporary relation is created instead, and used in body of outer level query

Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

□ Rewrite: **select ...**

from L_1

where P_1 and exists (select *

from L_2

where P_2)

□ To: **create table t_1 as**
select distinct V
from L_2
where P_2^1

select ...

from L_1, t_1

where P_1 and P_2^2

- P_2^1 contains predicates in P_2 that do not involve any correlation variables
- P_2^2 reintroduces predicates involving correlation variables, with relations renamed appropriately
- V contains all attributes used in predicates with correlation variables

Optimizing Nested Subqueries (Cont.)

- In our example, the original nested query would be transformed to
create table t_1 as
select distinct ID
from $teaches$
where $year = 2007$

select $name$
from $instructor, t_1$
where $t_1.ID = instructor.ID$

- The process of replacing a nested query by a query with a join (possibly with a temporary relation) is called **decorrelation**.
- Decorrelation is more complicated when
 - the nested subquery uses aggregation, or
 - when the result of the nested subquery is used to test for equality, or
 - when the condition linking the nested subquery to the other query is **not exists**,
 - and so on.

Materialized Views**

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
create view *department_total_salary*(*dept_name*,
total_salary) **as**
select *dept_name*, **sum**(*salary*)
from *instructor*
group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is **required frequently**
 - Saves the effort of finding multiple tuples and adding up their amounts

Materialized View Maintenance

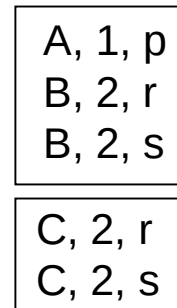
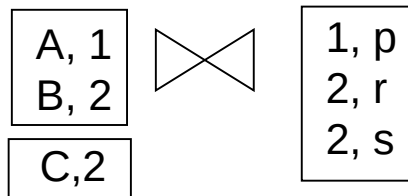
- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Above methods are directly supported by many database systems
 - ▶ Avoids manual effort/correctness issues

Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted $\mathbf{i}_r$ and $\mathbf{d}_r$
- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions

Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r :
 - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
 - And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
 - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$



Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$
- Projection is a more difficult operation
 - $R = (A,B)$, and $r(R) = \{ (a,2), (a,3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a,2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a,3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1
 - On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$
 - ▶ if the count becomes 0, we delete the tuple from $\Pi_A(r)$

Aggregate Operations (Cont.)

- **min, max:** $v = \mathcal{G}_{min(B)}(r)$.
- Handling insertions on r is straightforward.
- Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of r that are in the same group to find the new minimum

Aggregation Operations

- $\text{count} : v = {}_A \mathbf{g}_{\text{count}(B)}^{(r)}.$
 - When a set of tuples i_r is inserted
 - ▶ For each tuple r in i_r , if the corresponding group is already present in v , we increment its count, else we add a new tuple with count = 1
 - When a set of tuples d_r is deleted
 - ▶ for each tuple t in i_r , we look for the group $t.A$ in v , and subtract 1 from the count for the group.
 - If the count becomes 0, we delete from v the tuple for the group $t.A$
- $\text{sum} : v = {}_A \mathbf{g}_{\text{sum}(B)}^{(r)}$
 - We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
 - Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
 - ▶ Cannot simply test for $\text{sum} = 0$ (why?)
- To handle the case of **avg**, we maintain the **sum** and **count** aggregate values separately, and divide at the end

General Transformation Rules for Relational Algebra Operations:

1. Cascade of s: A conjunctive selection condition can be broken up into a cascade (sequence) of individual s operations:
$$s_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } cn}(R) = s_{c1}(s_{c2}(\dots(s_{cn}(R))\dots))$$
2. Commutativity of s: The s operation is commutative:
$$s_{c1}(s_{c2}(R)) = s_{c2}(s_{c1}(R))$$
3. Cascade of p: In a cascade (sequence) of p operations, all but the last one can be ignored:
$$p_{List1}(p_{List2}(\dots(p_{Listn}(R))\dots)) = p_{List1}(R)$$
4. Commuting s with p: If the selection condition c involves only the attributes A1, ..., An in the projection list, the two operations can be commuted:
$$p_{A1, A2, \dots, An}(s_c(R)) = s_c(p_{A1, A2, \dots, An}(R))$$

5. Commutativity of $\bowtie$ (and $\times$): The operation is commutative as is the $\times$ operation: $R \bowtie_c S = S \bowtie_c R$; $R \times S = S \times R$
6. Commuting s with $\bowtie$ (or $\times$): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R —the two operations can be commuted as follows :

$$s_c (R \bowtie S) = (s_c (R)) \bowtie S$$

Alternatively, if the selection condition c can be written as $(c1 \text{ and } c2)$, where condition $c1$ involves only the attributes of R and condition $c2$ involves only the attributes of S , the operations commute as follows:

$$s_c (R \bowtie S) = (s_{c1} (R)) \bowtie (s_{c2} (S))$$

7. Commuting p with $\bowtie$ (or x): Suppose that the projection list is $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, where $A_1, \dots, A_n$ are attributes of R and $B_1, \dots, B_m$ are attributes of S . If the join condition c involves only attributes in L , the two operations can be commuted as follows:

$$p_L (R \bowtie_C S) = (p_{A_1, \dots, A_n} (R)) \bowtie_C (p_{B_1, \dots, B_m} (S))$$

If the join condition c contains additional attributes not in L , these must be added to the projection list, and a final p operation is needed.

8. Commutativity of set operations: The set operations $\cup$ and $\cap$ are commutative but $-$ is not.
9. Associativity of $\bowtie$, $\times$, $\cup$, and $\cap$: These four operations are individually associative; that is, if θ stands for any one of these four operations (throughout the expression), we have

$$(R \theta S) \theta T = R \theta (S \theta T)$$
10. Commuting σ with set operations: The σ operation commutes with $\cup$, $\cap$, and $-$. If θ stands for any one of these three operations, we have

$$\sigma_c (R \theta S) = (\sigma_c (R)) \theta (\sigma_c (S))$$

11. The π operation commutes with $\cup$.

$$\pi_L (R \cup S) = (\pi_L (R)) \cup (\pi_L (S))$$

12. Converting a (σ, x) sequence into $\bowtie$: If the condition c of a σ that follows a x Corresponds to a join condition, convert the (σ, x) sequence into a $\bowtie_c$ as follows:

$$(\sigma_c (R \times S)) = (R \bowtie_c S)$$

13. Other transformations

5. Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.
6. Identify subtrees that represent groups of operations that can be executed by a single algorithm.

Top-K Queries

□ Top-K queries

```
select *  
from r, s  
where r.B = s.B  
order by r.A ascending  
limit 10
```

- Alternative 1: Indexed nested loops join with r as outer
- Alternative 2: estimate highest r.A value in result and add selection (**and** r.A $\leq$ H) to where clause
 - ▶ If < 10 results, retry with larger H

Optimization of Updates

□ Halloween problem

**update R set $A = 5 * A$
where $A > 10$**

- If index on A is used to find tuples satisfying $A > 10$, and tuples updated immediately, same tuple may be found (and updated) multiple times
- Solution 1: *Always defer updates*
 - ▶ collect the updates (old and new values of tuples) and update relation and indices in second pass
 - ▶ Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
- Solution 2: *Defer only if required*
 - ▶ Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.

Join Minimization

□ Join minimization

```
select r.A, r.B  
from r, s  
where r.B = s.B
```

□ Check if join with s is redundant, drop it

- E.g. join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s

- Other sufficient conditions possible

```
select r.A, s2.B  
from r, s as s1, s as s2  
where r.B=s1.B and r.B = s2.B and s1.A < 20 and s2.A < 10
```

- ▶ join with s1 is redundant and can be dropped (along with selection on s1)

- Lots of research in this area since 70s/80s!

Multiquery Optimization

- Example

Q1: **select * from (r natural join t) natural join s**

Q2: **select * from (r natural join u) natural join s**

- Both queries share common subexpression (r natural join s)
- May be useful to compute (r natural join s) once and use it in both queries

- ▶ But this may be more expensive in some situations
 - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper

- **Multiquery optimization**: find best overall plan for a set of queries, exploiting sharing of common subexpressions between queries where it is useful

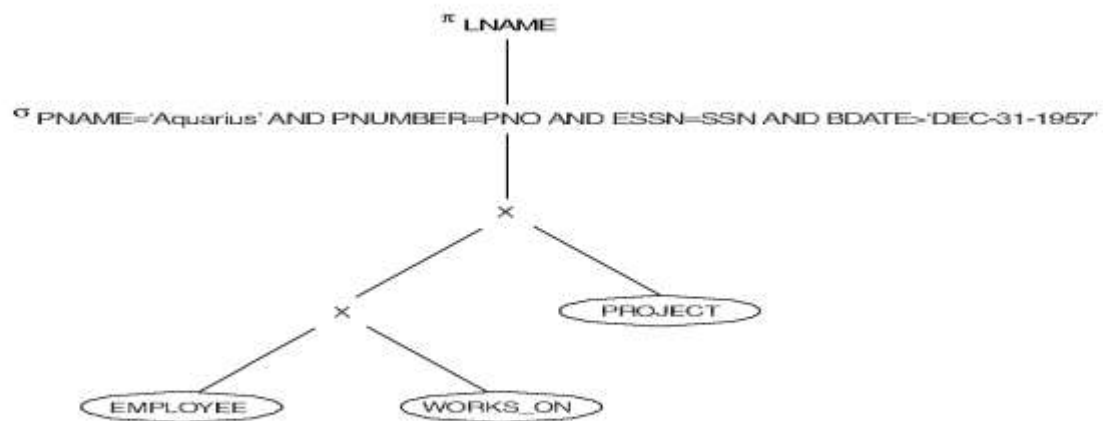
Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
 - optimize each query separately
 - detect and exploiting common subexpressions in the individual optimal query plans
 - ▶ May not always give best plan, but is cheap to implement
 - **Shared scans**: widely used special case of multiquery optimization
- Set of materialized views may share common subexpressions
 - As a result, view maintenance plans may share subexpressions
 - Multiquery optimization can be useful in such situations

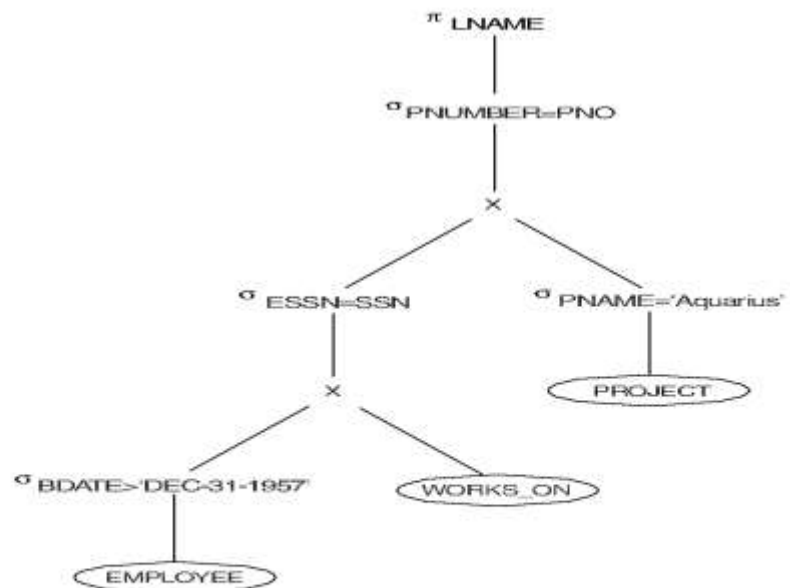
Parametric Query Optimization

- Example
select *
from r natural join s
where r.a < \$1
 - value of parameter \$1 not known at compile time
 - ▶ known only at run time
 - different plans may be optimal for different values of \$1
- Solution 1: optimize at run time, each time query is submitted
 - ▶ can be expensive
- Solution 2: **Parametric Query Optimization:**
 - optimizer generates a set of plans, optimal for different values of \$1
 - ▶ Set of optimal plans usually small for 1 to 3 parameters
 - ▶ Key issue: how to do find set of optimal plans efficiently
 - best one from this set is chosen at run time when \$1 is known
- Solution 3: **Query Plan Caching**
 - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
 - Implemented in many database systems

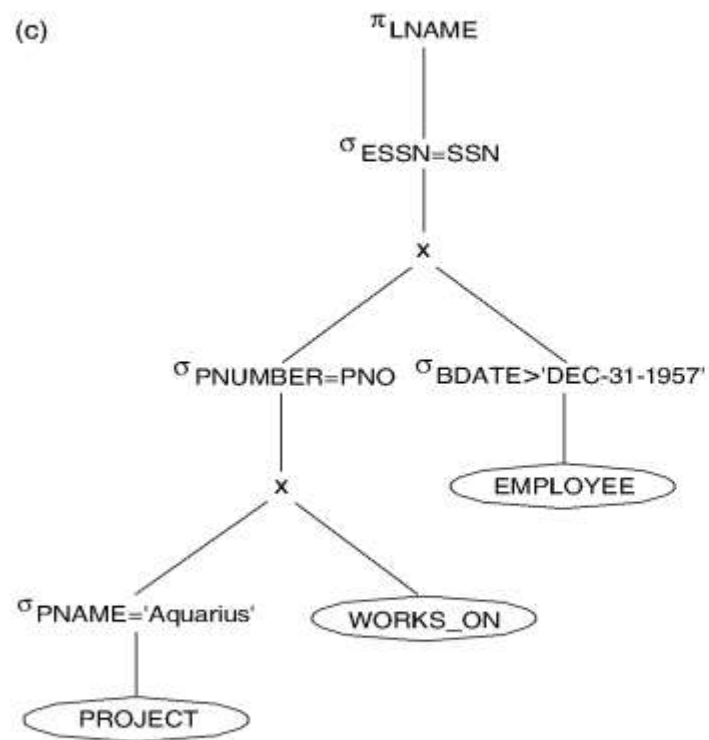
(a)



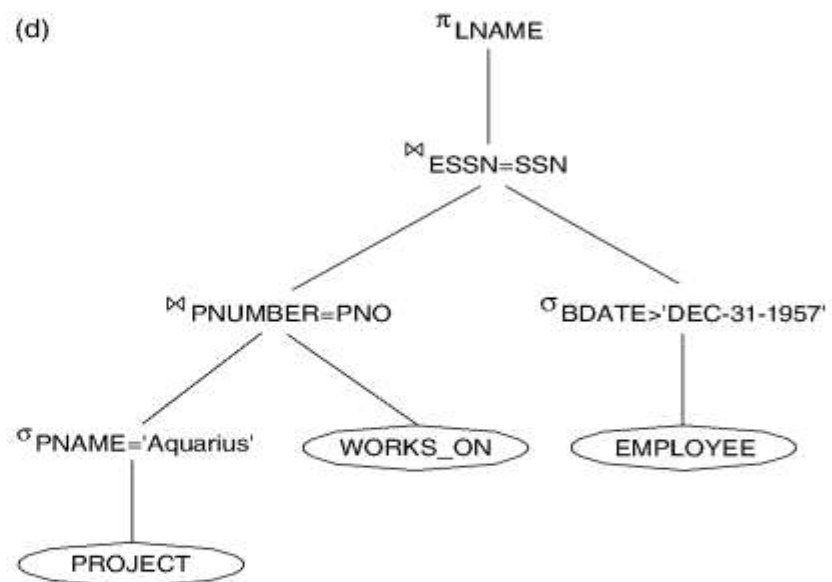
(b)



(c)



(d)



(e)

